SQL (Structured Query Language)
PostgreSQL

→ **Database**

It is an organised collection of data that is stored electronically so that the data can be stored, searched, retrieved, updated & managned efficiently.

↳ **In short** - Instead of keeping information randomly in files & database organizes it in a structured way.

↳ **Example:** A database can organise information into seperate tables like - A ecommerce Application

E-commerce Database

- customers
- Products
- orders
- payments

***Ex**

- Instagram → Users, Posts, comments
- Amazon → Products, customers, orders
- Banking → Accounts, transactions, customers
- Hospital → Patients, doctors, Appointments
- Our ML/API Projects → user, predictions, products etc.

↳ **Note** - Database = organized storage + efficient data management

→ DBMS (Database Management System)

It is software that allows users & Applications to create, store, retrieve, update, delete, organise & Manage data inside databases.

↳ Diff betⁿ Database Vs DBMS

The database contains the data, while the DBMS provides the system through which we interact with & manage that data.

↳ Examples

- PostgreSQL
- MySQL
- Oracle Database
- Microsoft SQL server
- SQLite

↳ DBMS Working

It can handle operations such as:

create database
↓
create tables
↓
Insert data
↓
Read data
↓
Update data
↓
Delete data

It can also handle:

- Authentication
- Permission
- Transactions
- Data Integrity
- concurrency
- Backup/recovery
- Query Preprocessing

→ RDBMS (Relational Database Management system)

It is a type of DBMS that stores data primarily in tables & allows relationships to be established betⁿ those tables using keys

- **Relational** means the database can represent relationships betⁿ different sets of data.

↳ **Example** : RDBMS stores data in multiple tables & connects those tables using relationships.

- **Like** - A college database

Students

student_id	name
1	Rahul
2	Aman

courses

courses_id	course_name
101	BCA
102	B.Tech

Enrollments

students_id	course_id
1	101
2	102

Here, student_id & course_id connect the tables.

→ SQL (structured Query Language)

It is a language used to communicate with & manipulate relational databases.

we use SQL to tell the databases what we want to do with the data.

↳ SQL is used for

- creating structures
- Adding Data
- Reading Data
- Updating Data
- Deleting Data

→ PostgreSQL

It is an open-source, object-relational Database Management (ORDBMS) that provides SQL functionality along with many advanced database features.

↳ Used for :

- Web Applications
- Backend systems
- Data Analysis
- API's
- Enterprise Applications
- Geospatial Applications
- Data-intensive systems

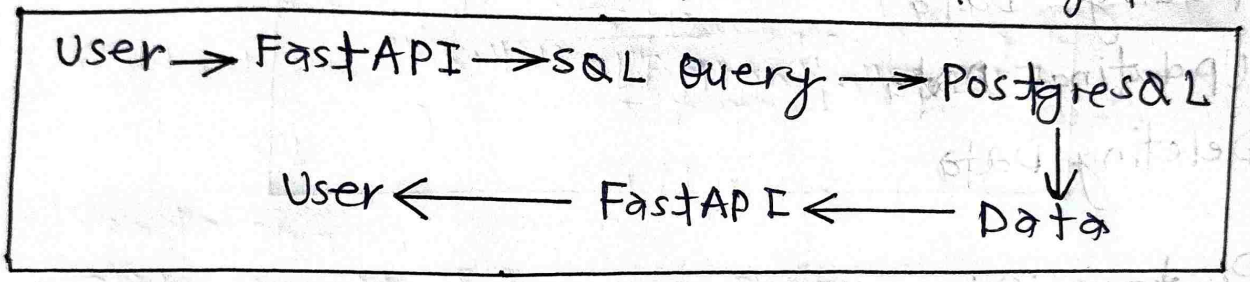
Why PostgreSQL ?

Because it provides:

- SQL support
- Transactions
- Foreign keys
- constraints
- complex queries
- JOIN's
- CTE's
- Window functions
- Indexes
- Views
- JSON / JSONB
- Extensions
- Advanced data types

Example

If make a application, it might look like



ACID (Atomicity, consistency, Isolation, Durability)

It refers to four properties that ensures database transactions are reliable, accurate & safe.

(A) Atomicity -

here a transaction is treated as a single unit. Either all operations are completed or None are.

- Example- Bank Transfer
  (1) 1000 is deducted from Aniket.
  (2) 1000 is added to Baby.

  If step (2) fails, step (1) rolledback. Mean Aniket got returned IK.

In short- All or Nothing

Consistency

Here, a transaction must keep the database in a valid state & follow all defined rules & constraints.

- **Example** - A PRIMARY KEY cannot contain duplicate values.

- **Note** - Valid Data

Isolation

Here, when multiple transactions run at the same time, each transaction is controlled so that they do not improperly interfere with each other.

- **Note** - Transactions stay separated

Durability

Once a transaction is successfully committed, it changes remain saved even if the database crashes or the system loses power.

- **Example** -

INSERT → COMMIT → system crashes
↓
Data remains

→ NoSQL

It refers to database systems that use data model other than the traditional relational table model, often providing flexible schemas & model such as -

- Documents
- Key-value pairs
- graphs
- wide columns

↳ **Example** : MongoDB stores documents similiar to :

```
{
  "name" : "Rahul",
  "age" : 21,
  "skills" : ["Python", "SQL"]
}
```

↳ SQL vs NoSQL vs PostgreSQL

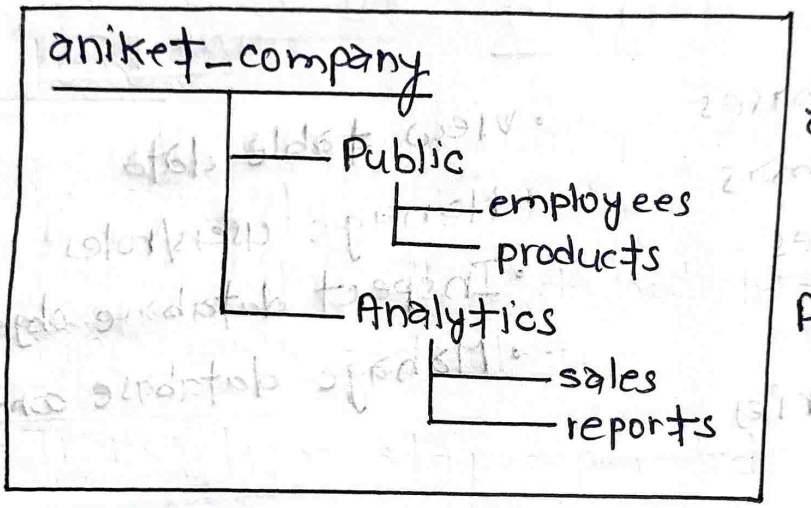
Feature	SQL	NoSQL	PostgreSQL
What is it?	Query Language	Database Approach	RDBMS / database system
Data	Works with relational Databases	Documents, Key-values, graphs etc	Tables (rows & columns)
Schema	Usually structured	Usually flexible	structured
Relationships	supported by relational databases	Depends on database	strong support
Example	SELECT * FROM users;	MongoDB, Redis	SELECT * FROM users;

→ Schema

It's a logical namespace/container inside a PostgreSQL database that organizes database object such as tables, views, functions, sequences & types.

Short:- schema is like a folder inside a database.

↳ Example



Here:-
aniket_company → Database
Public, Analytics → schema

↳ Default schema

PostgreSQL commonly provides `Public` as the default schema.

→ Table, column & Row

↳ **Table**: It is a database object that stores structured data in rows & column.

↳ **column**: It represents a specific attribute/field of a table

↳ **Row**: It represents one complete record in a table.

→ PgAdmin

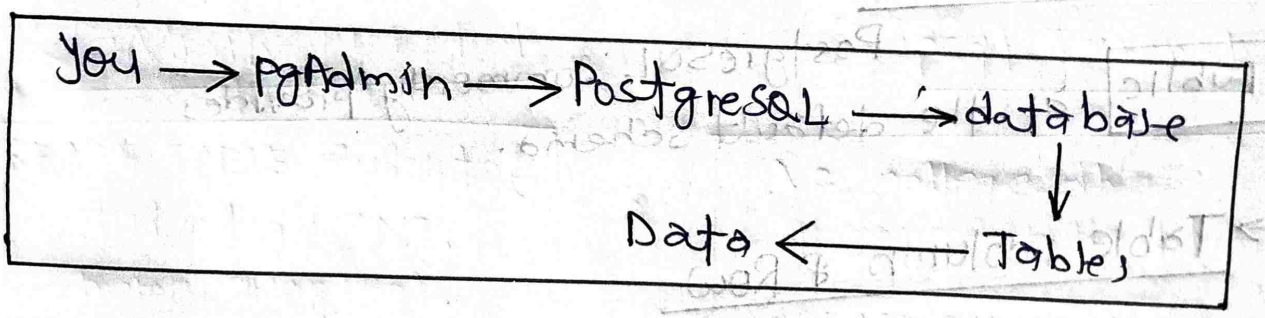
It is a graphical administration & development tool for PostgreSQL.

It provides a GUI through which we can interact with PostgreSQL without having to perform every task through the command line.

↳ Use :-

- create databases
- create schemas
- create tables
- write SQL
- Execute queries
- view table data
- Manage users/roles
- Inspect database objects
- Manage database configurations

↳ Flow



→ Query Tool

In PgAdmin it is an SQL editor & execution environment used to write & execute SQL statements against a selected PostgreSQL database.

↳ Ex

```
SELECT * FROM employees;
```

↳ **Used for**

- CREATE
- INSERT
- SELECT
- UPDATE
- DELETE
- ALTER
- DROP

* for advanced SQL

- JOIN
- GROUP BY
- CTE
- WINDOW FUNCTIONS
- SUBQUERIES.

→ **PostgreSQL server**

It is the running PostgreSQL database service/process that accepts connections & Manages databases, Queries, transactions & stored data.

→ **psql**

It is PostgreSQL's interactive command-line client.

Instead of using a GUI like PgAdmin, we can interact with PostgreSQL directly in terminal.

↳ **Ex-**

first :- `psql -U postgres` in terminal to execute psql

then :- `SELECT * FROM students;`

→ CRUD operations

* Basically stands for -

C - create
R - Read
U - update
D - Delete

→ Create Database -

For PdAdmin (Query Tool)

```
CREATE DATABASE college_db;
```

For psql (terminal)

```
CREATE DATABASE college_db;
\l # list all database
\c college_db #connect
   to college_db
```

→ Create Table

PgAdmin - Query Tool

```
CREATE TABLE students(
    student_id INT,
    name CHAR(50),
    age INT,
    grade CHAR(1)
);
```

psql Terminal

```
\c college_db
#connect to the database

CREATE TABLE students(
    student_id INT,
    name CHAR(50),
    age INT,
    grade CHAR(1)
    );

\dt
#List tables
```

→ INSERT - Add Data

PgAdmin - Query Tool

```
INSERT INTO students(
    name, age, grade)
VALUES ('Aniket', 20, 'A');

#Insert Multiple Rows
INSERT INTO students(
name, age, grade)
VALUES
('Akarsh', 23, 'A'),
('Aniket', 20, 'B');
#Insert values into
  all columns
INSERT INTO students
VALUES
(3, 'Raj', 23, 'C');
```

Psql Terminal

same code for Both

→ SELECT - Read Data

PgAdmin (Query Tool)

```
SELECT * FROM students;
#Read all columns & rows

SELECT name FROM students;
#Read only the name column
SELECT *
FROM students
WHERE age=23;
#Read records where age 23
```

Psql Terminal

same code for Both

→ UPDATE - Modify Data

PgAdmin (Query Tool)

```
UPDATE students
SET age = 24
WHERE name = 'Aniket';
```

we can do without "WHERE" for every row can be updated:

```
UPDATE students
SET age = 24;
```

psql Terminal

same for both

→ DELETE - Remove Data

PgAdmin (Query Tool)

```
DELETE FROM students
WHERE name = 'Babu';
#Delete specific record

DELETE FROM students;
#without WHERE, all rows
are deleted:
```

psql Terminal

same both

→ Problems with Basic Tables

(1) NULL Values :- A column can contain NULL when value is provided.

This can leave important information missing like student-id etc.

Problem - Required fields can be left empty.

Solution - Use "NOT NULL" to require a value.

(2) Duplicate value : A Normal column does not automatically prevent the same value from being inserted multiple times.

Problem - Identifier such as "student_id" can contain duplicates.

Solution - Use "PRIMARY KEY" or "UNIQUE" to prevent unwanted Duplicates.

(3) Invalid values

A Data type only controls the type of values, not whether the value makes logical sense

Problem - An "INT" column can accept invalid values such as negative or unrealistic ages.

Solution - Use "CHECK" to enterce valid condns.

→ Data Types & Constraints

(1) Numeric Data Types -

Data Type	Use	Example	Size
SMALLINT	small whole numbers	Age, Quantity	2 bytes integer (-32,768 to 32,767)
INT/INTEGER	Normal whole numbers	ID, count	4 bytes integer (-2 Billion to 2B)
BIGINT	very large whole numbers	Large ID's, view count	8 Bytes integer
NUMERIC (P,S) / DECIMAL (P,S)	Exact decimal values	Price, salary	for exact precision limit can set
REAL	single-precision decimal	scientific / approximate values	4-byte floating point
DOUBLE PRECISION	Higher-precision decimal	scientific calculations	8-byte floating point
SERIAL	Auto-increment integer	Primary key	as long serial goes

→ Example :-

Psql Terminal

{ almost same for PgAdmin just don't "\" commands

```
\c college_db
#connect to our database

CREATE TABLE numeric_demo (
    small_number SMALLINT,
    normal_number INT,
    large_number BIGINT,
    price NUMERIC(10,2),
    measurement REAL,
    precise_value DOUBLE PRECISION,
    id SERIAL
);
#for create a Table demonstrating numeric types

INSERT INTO numeric_demo (
    small_number,
    normal_number,
    large_number,
    price,
    measurement,
    precise_value)
VALUE (
    100,
    50000,
    9000000000,
    1499.99,
    30.5
    12345.678901
);
#INSERT example values
SELECT * FROM numeric_demo; #View data
\dt  # List tables
```

↳ Numeric(P,S) / Decimal(P,S)

- **Ex** : Numeric(10, 2)

  10 → total no. of digits

  2 → digits After decimal

  Ex = 14999999.99

  8 digits + 2 decimal digit = 10

- **code Example**

PgAdmin / Psql Terminal

```
CREATE TABLE products_price (
    product_id INT,
    price NUMERIC(10, 2)
);
#create Table with exact decimal price
INSERT INTO products_price
VALUES (1, 1499.99);
#Insert price
SELECT * FROM products_price;
```

↳ SERIAL

It Automatically generates sequential integer values when a new row is inserted.

Note- It is a PostgreSQL-specific shorthand for an auto-incrementing integer mechanism.

- Example code

PgAdmin / Psql

```
CREATE TABLE users (
    user_id SERIAL,
    name VARCHAR(50)
    );
#serial automatically generates
  the ID
INSERT INTO users(name)
VALUES ('Anshu');
INSERT INTO users (name)
VALUES ('yashmine');
#No user-id required
SELECT * FROM users;
```

(2) Character / string Data Types

Type	Mean	Example
CHAR(n)	Fixed-length string	CHAR(5)
VARCHAR(n)	Variable-length string with max^m length	VARCHAR(50)
TEXT	Variable-length text (here max^m length)	Description/Bio (its without a declared character limit)

↳ Example

```
CREATE TABLE text_demo (
    fixed_code CHAR(5),
    username VARCHAR(50),
    description TEXT
);
```

```
INSERT INTO text_demo
VALUES (
    'ABC'
    'Baby'
    'this is the boilogy data or
     desription.'
    );

SELECT * FROM text_demo;
```

(3) Boolean : It stores logical values:

```
TRUE , FALSE , NULL
```

↳ **Ex** **PgAdmin / Psql**

```
CREATE TABLE products_status (
    product_id INT,
    product_name VARCHAR (50),
    is_available BOOLEAN
  );

INSERT INTO products_status
VALUES
(1 , 'Laptop' , TRUE )
(2 , 'Mouse' , FALSE);

SELECT * FROM products_status
```

Constraints

constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each row	id SERIAL PRIMARY KEY
NOT NULL	column must have a value	name TEXT NOT NULL
UNIQUE	Prevents duplicate values	email TEXT UNIQUE
DEFAULT	Provides a vaule Automatically	created_at TIMESTAMP DEFAUT now()
CHECK	Validates a cond^n	age INT CHECK (age >8)
FOREIGN KEY	Links two tables	user_id INT REFERENCES users(id)

Ex

Psql / PgAdmin

```
\c college_db  (only for psql terminal)
CREATE TABLE users ( #Parent Table
  id SERIAL PRIMARY KEY, 1
  #id Uniquely Identifies each usr
  name TEXT NOT NULL,
  # Name cannot be empty
  email TEXT UNIQUE,
  #Email cannot be Duplicated
  age INT CHECK(age > 18)., # Must greter than 18
  created_at TIMESTAMP DEFAULT
  now()  ); # Automatically stores current date/time
```

```
CREATE TABLE orders
#child Table & connects each other to a User
    Order_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL
    #Must contain a UserID
    FOREIGN KEY (user_id)
        REFERENCES users(id)
);

INSERT INTO users (name, email, age)
#Insert a valid User
VALUES ('Akarsh', 'ak@.com', 23);

INSERT INTO orders (user_id)
VALUES (1);
#Insert an order for that User ID 1

SELECT * FROM users;
SELECT * FROM orders;   # view tables
\dt (for terminal only)
```

→ Clauses

clause	Purpose
SELECT	choose which columns to display
FROM	specify the table
WHERE	Filter rows based on a cond^ns
GROUP BY	Group rows for aggregation
HAVING	Filter aggregated groups

Clause	Purpose
ORDER BY	sort results Ascending/descending
LIMIT	Limit no. of rows returned
As	Rename columns / tables temporarily
DISTINCT	Return only unique values

↳ Example- PgAdmin/psql

```
\c college-db  (for only Terminal)

SELECT name, price
FROM products;
# SELECT → choose columns to display

SELECT * FROM products;
# FROM → specify the table

SELECT * FROM products WHERE price > 1000;
#WHERE → filter rows based on a cond^n

SELECT category, COUNT(*) AS total_products
FROM products
GROUP BY category;
#GROUP BY → group rows for aggregation

SELECT category, COUNT(*) AS total_products
FROM products
GROUP BY category
HAVING COUNT(*) > 1;
#HAVING → filter Aggregated groups
```

```
SELECT * FROM products ORDER BY price DESC;
# ORDER BY → sort results
# ASC = ascending , DESC = descending

SELECT * FROM products ORDER BY price ;
#As Default it is Ascending

SELECT * FROM products LIMIT 5;
#LIMIT → limit returned rows

SELECT name AS product_name ,
price AS product_price FROM products ;
# AS → temporarily rename a column

SELECT DISTINCT category FROM products;
#DISTINCT → return unique value only

Idt #List table (for only termined)
```

→ **Clauses - Operators**

operator Type	operators	Purpose	Easy Meaning
Comparison	=, !=, <, >, <=, >=	Compare value	Compare
Range	BETWEEN	check whether a value is within a range	Between
Set	IN	check whether a value matches any value in a List	Any of these
Pattern	LIKE	Search for a text Pattern	Pattern match

Logical	AND, OR, NOT Both ↙ either ↙ opposite ↙	combine or reverse condns	combine condns

↳ Wildcards

(1) % → Matches zero or more characters

Ex - `LIKE 'A%'` → starts with A

(2) _ → Matches exactly one character

Ex - `LIKE 'A_'` → A + one character

↳ Examples

PgAdmin / Psql

```
\c college_db (for terminal)

SELECT * FROM products WHERE price = 1000;
#comparison operator

SELECT * FROM products WHERE price != 1000;
#comparison operator etc

SELECT * FROM products WHERE price BETWEEN
500 AND 1000;
#Between → value within range

SELECT * FROM products WHERE category IN (
'Electronics', 'Fitness');
#IN → match any value from the list
```

```
SELECT * FROM products WHERE name LIKE 'A%';
#LIKE → search text pattern

SELECT * FROM products WHERE name LIKE '_p%';
#LIKE → search text pattern

SELECT * FROM products WHERE price > 1000
 AND category = 'Electronics';
#AND → all condns must be True

SELECT * FROM products WHERE category =
'Electronics' OR category = 'Fitness';
#OR → at least one condn must be true

SELECT * FROM products WHERE NOT
 category = 'Electronics';
```

→ Aggregation Functions

Function	Purpose	Example
COUNT()	counts rows/values	Total products
SUM()	Adds numeric values	Total stock
AVG()	calculates Average	Average price
MIN()	Finds smallest value	cheapest Product
MAX()	Finds highest value	Most expensive product

Note- COUNT(*) counts rows, while COUNT(column) counts non-NULL values in that column.

↳ Example -

PgAdmin/Psql

```
\c college_db  (for only Terminal)

SELECT COUNT(*) AS total_products FROM
products;
#COUNT() → count total rows
SELECT SUM(stock) AS total_stock FROM
products;
#SUM() → calculate total
SELECT AVG(price) AS average_price FROM
products;
#AVG() → calculate Average
SELECT MIN(price) AS cheapest_product FROM
products;
#MIN() → find smallest value
SELECT MAX(price) AS most_expensive_product
FROM products;
#MAX() → find highest value
=========
#With GROUP BY
=========
SELECT category, SUM(stock) AS total_stock
FROM products GROUP BY c
#Total stock for each category
SELECT category, AVG(price) As average_price
FROM products GROUP BY category;
#Average price for each category
\dt (for terminal)
```

→ String Functions

they are used to Modify, clean, format, combine or extract text from database columns.

Function	Purpose
UPPER()	converts text to uppercase
LOWER()	converts text to lowercase
LENGTH()	counts characters
SUBSTRING/ SUBSTR()	Extracts part of text
LEFT()	Takes characters from the Left
RIGHT()	Takes characters from the right
CONCAT()	Joins strings together
CONCAT_WS()	joins strings using a separator
TRIM()	Removes spaces from beginning/ end
REPLACE()	Replaces text with other text

Note-

CREATE TABLE → create structure
ALTER TABLE → Modify structure
DROP TABLE → Delete structure + data

→ Example -

PgAdmin/Psql

```
SELECT UPPER(name) FROM products;
#UPPER() → convert to uppercase
SELECT LOWER(name) FROM products;
#LOWER() → convert to lowercase
SELECT LENGTH(name) FROM products;
#LENGTH() → count character
SELECT SUBSTR(sku_code, 1, 2) FROM products;
#SUBSTR() → extract part of text
#start at position 1, take 2 character
SELECT LEFT(sku_code, 2) FROM products;
#LEFT() → take character from left
SELECT RIGHT(sku_code, 2) FROM products;
#RIGHT() → take characters from right
SELECT CONCAT(name, ' ', category) FROM products;
#CONCAT → join text
SELECT CONCAT_WS(' ; ', name, category) FROM
products;
#CONCAT_WS() → join using seperator
SELECT TRIM(' Hello ')
#TRIM() → remove leading/trailing spaces
SELECT REPLACE(sku_code, 'AB', 'GG')
FROM products;
#REPLACE() → replace text
```

→ ALTER TABLE

It is used to Modify the structure of an existing table without deleting the Table itself.

operation	Purpose	Example
ADD column	Add a new column	ALTER TABLE students ADD COLUMN email VARCHAR(100);
DROP column	Remove a column	ALTER TABLE students DROP COLUMN email;
RENAME column	Change column name	ALTER TABLE students RENAME COLUMN name To full_name;
Change data type	change column type	ALTER TABLE students ALTER COLUMN age TYPE SMALLINT;
SET DEFAULT	Add automatic default	ALTER TABLE students ALTER COLUMN age SET DEFAULT 18;
DROP DEFAULT	Remove default	ALTER TABLE students ALTER COLUMN age DROP DEFAULT;
ADD CONSTRAINT	Add a rule	ALTER TABLE students ADD CONSTRAINT age_check CHECK (age >=0);
DROP CONSTRAINT	Remove a rule	ALTER TABLE students DROP CONSTRAINT age_check;

operation	purpose	Example
RENAME TABLE	change table name	ALTER TABLE students RENAME TO school_students;

→ CASE in SQL

"CASE" is a conditional expression that works like if-else statements in programming. It checks cond^ns & return a result a result based on which cond^n is true.

↳ syntax

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
END
```

↳ Why use CASE?

(1) categorize data → create labels such as cheap/ Moderate/ Expensive

(2) create derived columns → Generate a result without changing the table

(3) Handle conditions → apply different results based on values

(4) Handle missing values → Display alternative values when needed

Note- It will create a temporary category or data use ALTER + UPDATE

↳ Example -

```
SELECT name , price
CASE
    WHEN price >1000 THEN 'Expensive'
    WHEN price BETWEEN 500
AND 1000 THEN 'Moderate'
    ELSE 'cheap'
END AS price_tag FROM products;
#create a temporary price category
```

↳ CASE with ALTER + UPDATE

This permanently stores the calculated result in a new column.

• Ex

```
ALTER TABLE products
ADD COLUMN price_tag NEXT ;
# Add a new column
UPDATE products
SET price_tag =
    CASE
        WHEN price > 1000 THEN 'expensive'
        WHEN price BETWEEN 500
    AND 1000 THEN 'Moderate'
        ELSE 'cheap'
    END;
```

Note-

SELECT + CASE → Temporary result
UPDATE + CASE → store result in table

→ Relationships in SQL

It connects data beth different tables using "keys".

↳ Primary Key Vs Foreign Key

Key	Purpose	In short
Primary key	Uniquely identifies a row	Identify
Foreign key	References another table's Primary key	connection

↳ Types of Relationships

Relationship	Mean	Example
one-to-one (1:1)	one Record ↔ one record	student ↔ student Profile
one-to-many (1:N)	One record → many records	student → Marks
Many-to-Many (M:N)	Many records ↔ many records	students ↔ course

↳ One-to-one Relationship (1:1)

one record in a Table A is associated with at most one record in Table B.

- **Why 1:1**

Example "student_id" is PRIMARY KEY in "student_profiles", so the same student ID cannot appear twice

- Example

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL;
#Parent Table
CREATE TABLE student_profiles (
    student_id PRIMARY KEY,
    address TEXT,
    age INT,
    phone VARCHAR(15),
#child Table
FOREIGN KEY (student_id) REFERENCES
student (student_id);
# connect to student Table
```

→ One-to-Many Relationship (Most Used)

One record in the parent table can be related to multiple records in the child Table.

- Example : one student have multiple subject marks:

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL);
#Parent Table
CREATE TABLE marks(
    mark_id SERIAL PRIMARY KEY
    student_id INT,
    subject VARCHAR(50),
    marks INT,
#Child Table
FOREIGN KEY(student_id) REFERENCE
students(student_id)); # connect maiks to students
```

Many to Many (M:N)

Many Record in one table can be related to many records in another table.

- **Example** : students ↔ courses
  - one student can enroll in many courses.
  - One course can have many students.

- **Why use Junction Table**

Because a direct M:N relationship is difficult to store, we use a junction (bridge) Table.

- **Example code :**

```
CREATE TABLE students(
    student_id SERIAL PRIMARY KEY,
    name VARCHAR (100) );
CREATE TABLE courses(
    course_id SERIAL PRIMARY KEY,
    course_name VARCHAR (100) );
CREATE TABLE student_courses (
    student_id INT REFERENCES students(student_id),
    course_id INT REFERENCES courses (course_id),
    PRIMARY KEY (student_id, course_id );
```

→ Joins in SQL

It combines data from two or more tables using a related column usually:

Primary key ↔ Foreign key

↳ Join Types

Join	Means
INNER JOIN	only matching rows from both Tables
LEFT JOIN	All left rows + matching right rows
RIGHT JOIN	All right rows + matching left rows
FULL JOIN	All rows from both tables
CROSS JOIN	Every possible combination

↳ Example -

```
SELECT
    s.student_id,
    s.name,
    m.subject,
    m.marks
FROM students s
INNER JOIN marks m
    ON s.student_id = m.student_id;
#INNER JOIN
```

Here we defined students as s & marks as m.

#LEFT JOIN

```
SELECT
    s.name,
    m.subject,
    m.marks
FROM students s
LEFT JOIN marks m
    ON s.student_id = m.student_id;
```

#RIGHT JOIN

```
SELECT
    s.name,
    m.subject,
    m.marks
FROM student s
RIGHT JOIN marks m
    ON s.student_id = m.student_id;
```

#FULL JOIN

```
SELECT
    s.name,
    m.subject
    m.marks
FROM students s
FULL JOIN marks m
    ON s.student_id = m.student_id;
```

#CROSS JOIN

```
SELECT
    s.name,
    m.subject
FROM students s
CROSS JOIN marks m;
```

→ Views

It is a virtual Table created from a saved SQL Query.

It does not normally store a seprate copy of the underlying data; when Queries PostgreSQL retreives the current data from the underlying tables.

↳ Why use Views ?

(1) Simplify complex Queries → save long queries

(2) Security → show only selected data

(3) Reusability → Reuse the same Query

(4) Abstraction → Hide Underlying table complexity

↳ Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

↳ Example

```
CREATE VIEW fitness_items AS
SELECT product_id, name, price, stock_quantity
FROM products
WHERE category = 'Fitness';
#create a view for fitness products
SELECT * FROM fitness_items;
#Read data from the view
```

Example: Managing Views

```
CREATE OR REPLACE VIEW fitness-items
AS
SELECT product_id, name, price
FROM products
WHERE category = 'Fitness';
#Modify/replace a view
DROP VIEW fitness_items;
#Delete a view
```

Procedures

It is a reusable block of SQL/PL/pgSQL logic stored inside the database & execute using CALL.

Why use Procedures?

(1) Reusability → write once, use many times

(2) Automation → Perform multiple operations

(3) security → control database operation

(4) Multi-step logic → Execute several SQL statement together.

Basic syntax

```
CREATE OR REPLACE PROCEDURE
procedure_name(Parameter datatype)
LANGUAGE plpsql
AS $$
BEGIN
END;
$$;
```

→ Example

```
#Add Product
CREATE OR REPLACE PROCEDURE
add_product (
    P_name VARCHAR(100),
    P_SKU CHAR(8),
    P_price NUMERIC(10,2),
    P_quantity INT,
    P_category TEXT)
LANGUAGE plpgsql
AS $$
BEGIN
#create a procedure

INSERT INTO products (name, sku_code, price,
stock_quantity, category)
VALUES (P_name, P_SKU, P_price, P_quantity,
P, category);
#Insert products into the Table
RAISE NOTICE 'Product %. Added sucessfully!'
P, name;
END;
$$;
=======
#call the Procedure
=======
CALL add_product ('Water bottle', 'B0123', 234.00,
45, 'Fitness');
```